Mini-GPT Médico

Relatório de estudo: da tokenização ao treino de um Transformer
com questões do Revalida e do Enare

Projeto em grupo, baseado no nanoGPT de Andrej Karpathy

Setembro de 2026

Resumo

Este relatório descreve, passo a passo, a construção de um modelo de linguagem do tipo GPT, escrito do zero em PyTorch e treinado com 5.632 questões de provas médicas brasileiras. O modelo prevê o próximo *caractere* de um texto. Cada etapa do projeto (dados e tokenização, arquitetura, treino, geração e experimentos) é apresentada em três camadas: a **teoria** que a justifica, com referência aos livros-texto; o **código** que a implementa; e os **números** obtidos. O modelo base, com 833.661 parâmetros, reduz a perda de validação de 4,90 para 1,25 nats por caractere em 13 minutos de CPU e aprende o formato das questões, mas não o conteúdo médico. Uma comparação honesta com modelos de n-gramas mostra que, com esse orçamento de treino, um modelo de 6-gramas (1,14) ainda o supera. Na Seção 8, com uma medida mais justa (janela deslizante), um modelo 5,8 vezes maior chega a 1,055 e passa o 6-grama; aumentar o contexto de 64 para 128 caracteres leva a perda a 1,015 e faz as alternativas geradas saírem em ordem em 74,6% dos casos, contra 42,7%.

Sumário

1 Como ler este relatório **3**

2 Fundamentos: o que é um modelo de linguagem **3**
2.1 Texto como sequência e a regra da cadeia 3
2.2 Entropia cruzada: a medida do erro 4
2.3 O ponto de partida: a perda de um modelo que não sabe nada 4
2.4 Softmax: de notas para probabilidades 4

3 Etapa 1: dados e tokenização **5**
3.1 O conjunto de dados 5
3.2 Limpeza: por que cada caractere importa 5
3.3 Separação em treino e validação 6
3.4 Tokenização em nível de caractere 7
3.5 Armazenamento e formação dos lotes 7

4 Uma régua de comparação: modelos de n-gramas **8**

5 Etapa 2: a arquitetura do Transformer **9**
5.1 Encoder, decoder e o que é o GPT 9
5.2 Visão geral e as dimensões 9
5.3 Embeddings de token e de posição 9
5.4 Atenção causal com várias cabeças 10
5.5 MLP, conexões residuais e LayerNorm 11
5.6 A saída do modelo 11

5.7 Contando os parâmetros . . . 12
5.8 Inicialização e a perda inicial . . . 12
5.9 Testes de sanidade da arquitetura . . . 13

6 Etapa 3: o treino **13**
6.1 O laço de treino . . . 13
6.2 O otimizador AdamW . . . 14
6.3 Dropout . . . 14
6.4 Avaliação durante o treino . . . 14
6.5 Resultados do modelo base . . . 14

7 Etapa 4: geração e avaliação **16**
7.1 Amostragem auto-regressiva . . . 16
7.2 Temperatura e top-k . . . 16
7.3 Resultados . . . 16

8 Etapa 5: experimentos **17**
8.1 Experimento 1: um modelo maior . . . 18
8.2 Uma correção na forma de medir . . . 20
8.3 Experimento 2: mais contexto . . . 20
8.4 Próximos experimentos sugeridos . . . 21

9 Resumo e próximos passos **21**

1 Como ler este relatório

O projeto tem cinco etapas, e cada uma virou um arquivo do repositório:

Etapa	O que faz	Arquivo	Seção
1. Dados e tokenização	texto → números	`data/prepare.py` `char_tokenizer.py`	3
2. Arquitetura	números → probabilidades	`model.py`	5
3. Treino	ajusta os pesos	`train.py`	6
4. Geração e avaliação	probabilidades → texto	`generate.py`	7
5. Experimentos	compara configurações	`configs/`, `compare.py`	8

A Figura 1 mostra o caminho completo. Ao longo do texto aparecem três tipos de caixa: **Teoria** (o que os livros dizem), **No código** (onde a ideia aparece no projeto) e **Atenção** (armadilhas e erros que de fato aconteceram). Cada seção termina com perguntas para conferir o entendimento.

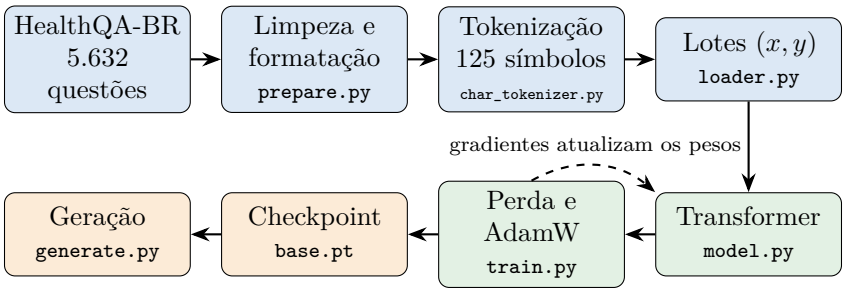


Figura 1: O caminho dos dados no projeto. Em azul, a Etapa 1; em verde, as Etapas 2 e 3; em laranja, a Etapa 4.

2 Fundamentos: o que é um modelo de linguagem

2.1 Texto como sequência e a regra da cadeia

Um texto é uma sequência de símbolos $x_1, x_2, \ldots, x_T$. No nosso caso cada símbolo é um caractere. Um *modelo de linguagem* atribui uma probabilidade a cada sequência. Pela regra da cadeia da probabilidade, que vale para qualquer distribuição,

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1}). \tag{1}$$

Essa igualdade transforma um problema difícil (dar probabilidade a um texto inteiro) em muitos problemas iguais e mais simples: **prever o próximo símbolo, dado o que veio antes**. Um modelo que faz isso é chamado de *auto-regressivo*. O GPT é exatamente isso: uma função que recebe um contexto e devolve uma distribuição de probabilidade sobre o próximo caractere.

Teoria. Regra da cadeia e modelos auto-regressivos

A regra da cadeia está em [1, cap. 3]. Jurafsky e Martin [2, cap. 3] usam a Equação (1) como ponto de partida para os modelos de n-gramas. O que muda entre um modelo de n-gramas e um Transformer é *como* se estima $P(x_t \mid x_{<t})$. O n-grama conta ocorrências e só olha os $n-1$ símbolos anteriores. O Transformer usa uma rede neural e olha até `block_size` símbolos para trás (64, no nosso caso).

2.2 Entropia cruzada: a medida do erro

Para cada posição t, o modelo produz um vetor de probabilidades $\hat{\boldsymbol{p}}_t \in \mathbb{R}^V$ sobre os $V = 125$ caracteres. O caractere correto é y_t. A perda usada no treino é a **entropia cruzada**:

$$\mathcal{L} = -\frac{1}{N}\sum_{t=1}^{N} \ln \hat{p}_t(y_t). \tag{2}$$

Em palavras: a média, sobre todas as posições, de menos o logaritmo da probabilidade que o modelo deu ao caractere certo. Se o modelo dá probabilidade 1 ao certo, a parcela é $-\ln 1 = 0$; se dá probabilidade quase zero, a parcela explode.

Teoria. Máxima verossimilhança e entropia cruzada

Minimizar (2) é o mesmo que **maximizar a verossimilhança** dos dados de treino: o produto (1) das probabilidades que o modelo dá ao texto real. Tomar o logaritmo transforma o produto em soma, e o sinal de menos transforma máximo em mínimo. Goodfellow et al. [1, seção 5.5] mostram que isso equivale a minimizar a divergência de Kullback–Leibler entre a distribuição dos dados e a do modelo. A unidade de (2) é o *nat*, porque usamos ln. Dividindo por $\ln 2$, obtemos *bits por caractere* [3, cap. 2].

Duas formas de interpretar um valor de perda.

- **Probabilidade média no caractere certo:** $e^{-\mathcal{L}}$ é a média geométrica das probabilidades $\hat{p}_t(y_t)$. Com $\mathcal{L} = 1{,}25$, isso dá $e^{-1{,}25} \approx 0{,}29$: o modelo põe, em média, 29% de probabilidade no caractere certo.
- **Perplexidade:** $\text{PPL} = e^{\mathcal{L}}$. Com $\mathcal{L} = 1{,}25$, $\text{PPL} \approx 3{,}5$: o modelo fica tão incerto quanto alguém escolhendo ao acaso entre 3 ou 4 caracteres igualmente prováveis. Jurafsky e Martin [2, cap. 3] usam a perplexidade como principal métrica de modelos de linguagem.

2.3 O ponto de partida: a perda de um modelo que não sabe nada

Um modelo recém-criado, sem informação nenhuma, deveria dar a mesma probabilidade $1/V$ a todos os caracteres. Nesse caso, cada parcela de (2) vale $-\ln(1/V) = \ln V$, e

$$\mathcal{L}_{\text{inicial}} = \ln V = \ln 125 \approx 4{,}83 \text{ nats}.$$

Esse número é o nosso **primeiro teste de sanidade**: se a primeira perda medida for muito diferente de 4,83, há erro na inicialização ou na ligação entre as camadas. Nosso modelo começou em 4,90 (Seção 5.8).

2.4 Softmax: de notas para probabilidades

A rede não produz probabilidades diretamente. Ela produz um vetor de *logits* $\boldsymbol{z} \in \mathbb{R}^V$, uma "nota" real para cada caractere. A função softmax transforma as notas em probabilidades:

$$\text{softmax}(\boldsymbol{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}. \tag{3}$$

Os valores ficam entre 0 e 1 e somam 1. Somar a mesma constante a todos os logits não muda o resultado. Com softmax e entropia cruzada juntos, o gradiente da perda em relação aos logits tem uma forma muito simples [1, seção 6.2.2.3]:

$$\frac{\partial \mathcal{L}_t}{\partial \boldsymbol{z}} = \hat{\boldsymbol{p}} - \boldsymbol{e}_y, \tag{4}$$

em que $\boldsymbol{e}_y$ é o vetor com 1 na posição do caractere certo e 0 nas demais. O treino "empurra para cima" a nota do caractere certo e "empurra para baixo" as outras, em proporção à probabilidade que cada uma recebeu.

Para verificar o entendimento

1. Se o vocabulário tivesse 60 caracteres, qual seria a perda inicial esperada?
2. Um modelo tem perda de validação 1,81 bits por caractere. Quanto é isso em nats? E a perplexidade?
3. Derive (4) a partir de (2) e (3). Dica: derive $\ln \text{softmax}(\boldsymbol{z})_y$ em relação a z_i, separando os casos $i = y$ e $i \neq y$.

3 Etapa 1: dados e tokenização

3.1 O conjunto de dados

Usamos o HealthQA-BR [21]: 5.632 questões de múltipla escolha do Revalida, do Enare Residência Médica e do Enare Multiprofissional, em português, com licença CC-BY 4.0. O `prepare.py` transforma cada questão num bloco de texto com um cabeçalho:

```
### Enare Residência Médica 2024 | Neurocirurgia
Existem vários tipos de osteotomia para o manejo do desequilíbrio sagital e
coronal da coluna vertebral. [...]
(A) 4-8 graus;
(B) 15-20 graus;
[...]
Gabarito: A
```

O cabeçalho `### Prova Ano | Área` dá estrutura ao corpus. Como todo bloco começa assim, o modelo aprende que depois de `### Revalida` vem um ano, depois uma área, depois um enunciado. Na geração, basta dar esse começo para o modelo escrever uma questão.

3.2 Limpeza: por que cada caractere importa

Com tokenização por caractere, cada símbolo distinto vira uma classe que o modelo precisa aprender a prever. Símbolos inúteis ou duplicados desperdiçam capacidade do modelo e dados de treino. A função `limpar` aplica, nesta ordem:

1. **Normalização Unicode NFC.** Em Unicode, "é" pode ser escrito como um caractere só (U+00E9) ou como "e" seguido de um acento combinante (U+0065 U+0301). Visualmente são iguais, mas para o tokenizador seriam sequências diferentes. A forma NFC junta tudo na versão de um caractere.
2. **Substituições de símbolos equivalentes.** Aspas curvas viram retas, travessões viram hífen, espaços especiais viram espaço comum, subscritos viram dígitos ($O_2 \rightarrow$ O2, como no resto do corpus).
3. **Homóglifos.** O texto original foi copiado de PDFs e contém letras gregas e cirílicas *visualmente idênticas* às latinas: "Allium cepa" com "p" cirílico (U+0440), alternativa "E" escrita com épsilon grego (U+0395). Sem tratamento, "o" latino (U+006F) e "o" cirílico (U+043E) seriam dois tokens diferentes, e o modelo teria que aprender duas vezes a mesma letra. O mapeamento `HOMOGLIFOS` troca 27 desses caracteres pelos latinos.
4. **Espaços e quebras de linha repetidos** são reduzidos.

5. **Formato das alternativas.** No dataset, as alternativas vinham como dicionário Python (`'A': 'texto'`), às vezes quebradas em várias linhas. Uma expressão regular as converte em `(A) texto`, numa linha só.

6. **Caracteres raros** (menos de 3 ocorrências no corpus inteiro) são removidos. Foram 17, como `$`, `^`, `®`, ½ e `ñ`. Um símbolo que aparece 1 ou 2 vezes não pode ser aprendido e só aumenta o vocabulário.

Resultado: o vocabulário caiu de 144 para **125 caracteres**. São 80 letras (maiúsculas, minúsculas e acentuadas), dígitos, pontuação e alguns símbolos médicos (°, ±, ≤, ≥, β, μ, $\rightarrow$).

Atenção. Um erro real que a limpeza revelou

O código da Etapa 1 foi escrito antes de ser rodado com o dataset verdadeiro. Ao rodar, apareceram os homóglifos e o formato de dicionário das alternativas, que ninguém tinha previsto. A lição: **sempre olhe os dados reais**, por exemplo imprimindo o vocabulário e procurando caracteres estranhos. Ver os 144 caracteres impressos foi o que mostrou letras gregas no meio de palavras em português.

3.3 Separação em treino e validação

Separamos 90% das questões para treino e 10% para validação. Dois cuidados:

- **Separação por questão, não por caractere.** Se cortássemos o texto contínuo num ponto qualquer, uma questão poderia ficar metade em cada lado, e parte da validação teria sido "vista" no treino. Isso é *vazamento de dados*, e faria a perda de validação parecer melhor do que é.
- **Embaralhar antes de separar.** As questões vêm ordenadas por prova e ano. Sem embaralhar, a validação teria só as provas mais recentes, e mediríamos a capacidade de generalizar para outro ano, não para questões novas do mesmo tipo.

Teoria. Por que precisamos de um conjunto de validação

O objetivo não é ir bem no treino: é ir bem em dados *não vistos*. Goodfellow et al. [1, seção 5.2] chamam isso de *erro de generalização*. A perda de treino mede o quanto o modelo se ajusta aos dados que viu. A perda de validação estima o erro em dados novos. Quando a de treino continua caindo e a de validação para de cair ou sobe, o modelo começou a *decorar* (overfitting).

Medida	Valor
Questões (treino / validação)	5.069 / 563
Caracteres de treino / validação	4.574.978 / 508.218
Tamanho do corpus	5,25 MB
Vocabulário	125 caracteres
Perda inicial esperada, $\ln 125$	4,83 nats
Caracteres mais frequentes	espaço (14,2%), a (9,9%), e (8,8%), o (7,3%)

Tabela 1: O corpus depois do `prepare.py`.

3.4 Tokenização em nível de caractere

Definição 1. *Um* tokenizador *é um par de funções:* `encode`*, que leva texto a uma lista de inteiros (tokens), e* `decode`*, que faz o caminho inverso, com* $\texttt{decode}(\texttt{encode}(s)) = s$.

Nosso tokenizador é o mais simples possível. Ele ordena os caracteres distintos do corpus e numera de 0 a 124:

```
class CharTokenizer:
    def __init__(self, chars):
        self.chars = sorted(set(chars))   # ordenar: mesmo corpus, mesmo vocabulário
        self.stoi = {ch: i for i, ch in enumerate(self.chars)}  # caractere -> inteiro
        self.itos = {i: ch for i, ch in enumerate(self.chars)}  # inteiro -> caractere

    def encode(self, text):
        return [self.stoi[ch] for ch in text]

    def decode(self, ids):
        return "".join(self.itos[int(i)] for i in ids)
```

Por que caractere, e não palavra ou sub-palavra? Há um compromisso entre o tamanho do vocabulário e o comprimento das sequências:

Nível	Vocabulário	Sequência	Palavras nunca vistas
Caractere	pequeno (125)	longa	impossíveis: tudo é soletrável
Sub-palavra (BPE)	médio (milhares)	média	raras
Palavra	enorme (dezenas de milhares)	curta	frequentes

O GPT-2 e seus sucessores usam BPE (*byte-pair encoding*) [10], que junta os pares de símbolos mais frequentes até atingir um vocabulário de dezenas de milhares de tokens. Escolhemos caracteres porque o código fica curto e fácil de explicar, o vocabulário é pequeno (a camada de saída tem só 125 classes) e o modelo nunca encontra uma palavra desconhecida. O custo é que o modelo precisa aprender a *soletrar*, e o contexto de 64 tokens cobre só cerca de 10 palavras.

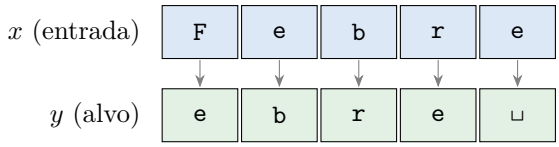
Atenção. O conflito de nomes com o pacote `tokenizer`

O arquivo se chamava `tokenizer.py`. Existe um pacote no pip com o mesmo nome (um tokenizador de islandês). Quando ele estava instalado e o código rodava fora da raiz do projeto, como num notebook aberto em `notebooks/`, o Python importava o pacote do pip no lugar do nosso e dava `ImportError: cannot import name 'CharTokenizer'`. A solução foi renomear o arquivo para `char_tokenizer.py`. Lição: evite nomes de módulo genéricos que podem coincidir com pacotes instalados.

3.5 Armazenamento e formação dos lotes

Os tokens são gravados em `train.bin` e `val.bin` como inteiros sem sinal de 16 bits (`uint16`, até 65.535, sobra muito para 125 símbolos). No treino, `np.memmap` lê o arquivo do disco sob demanda, sem carregar tudo na memória.

Um lote é formado sorteando `batch_size` posições iniciais e recortando `block_size`+1 tokens seguidos a partir de cada uma. A entrada x são os primeiros `block_size` tokens; o alvo y é o mesmo trecho **deslocado uma posição**:



Na posição t, o modelo vê $x_1, \ldots, x_t$ e precisa prever $y_t = x_{t+1}$.

Um bloco de 64 tokens gera **64 exemplos de treino ao mesmo tempo**: "depois de F, vem e", "depois de Fe, vem b", "depois de Feb, vem r", e assim por diante. Um lote de 32×64 gera 2.048 exemplos por passo. Isso só é possível porque a máscara causal (Seção 5.4) impede cada posição de ver as seguintes.

Para verificar o entendimento

1. Por que a normalização NFC precisa vir *antes* da contagem de caracteres raros?
2. Se não tivéssemos tratado os homóglifos, o que aconteceria com a perda de um modelo que encontra "o" cirílico na validação, mas nunca no treino?
3. Quantos exemplos de previsão existem num lote com `batch_size`= 16 e `block_size`= 128?

4 Uma régua de comparação: modelos de n-gramas

Antes de avaliar o Transformer, vale saber quanto um modelo *muito* mais simples consegue. O modelo de n-gramas, o modelo de linguagem clássico [2, cap. 3], supõe que o próximo caractere depende só dos $n-1$ anteriores (hipótese de Markov) e estima as probabilidades por contagem:

$$P(x_t \mid x_{t-n+1}, \ldots, x_{t-1}) \approx \frac{C(x_{t-n+1} \ldots x_t) + \alpha}{C(x_{t-n+1} \ldots x_{t-1}) + \alpha V}, \tag{5}$$

em que $C(\cdot)$ conta ocorrências no treino e α é uma suavização aditiva, que evita probabilidade zero para sequências nunca vistas. Contamos no `train.bin` e medimos a entropia cruzada no `val.bin`:

n	1	2	3	4	5	6	7
Perda de validação (nats), $\alpha = 0{,}01$	3,24	2,39	1,92	1,49	1,22	**1,14**	1,19

Tabela 2: Modelos de n-gramas por contagem. $n = 1$ só conhece a frequência de cada caractere; $n = 2$ olha o caractere anterior.

Duas lições que voltam mais adiante:

- **Mais contexto ajuda, até certo ponto.** A perda cai de 3,24 ($n = 1$) para 1,14 ($n = 6$). Com $n = 7$ ela volta a subir: a maioria das sequências de 7 caracteres da validação nunca apareceu no treino, e as contagens ficam esparsas demais. Esse é o problema que os modelos neurais resolvem: em vez de decorar sequências exatas, aprendem representações que *generalizam* para sequências parecidas [8].
- **A régua é exigente.** Um 6-grama bem suavizado chega a 1,14 nats. Um detalhe torna essa régua um pouco otimista: escolhemos α olhando a própria validação. O Transformer da Seção 6 precisa ser comparado com esse número.

5 Etapa 2: a arquitetura do Transformer

5.1 Encoder, decoder e o que é o GPT

O Transformer original [5] foi criado para tradução e tem duas torres (Figura 2). O *encoder* lê a frase de entrada ("That is good") com atenção bidirecional: cada palavra vê todas as outras. O *decoder* escreve a tradução ("Das ist gut") uma palavra por vez, com atenção mascarada, e consulta o encoder pela *atenção cruzada*.

O GPT [6] não traduz: ele continua um texto. Não há frase de entrada para o encoder ler, então o GPT usa **só o decoder, sem a atenção cruzada**. Cada bloco tem apenas atenção mascarada e MLP. Nosso modelo é exatamente isso. Bishop e Bishop [4, cap. 12] apresentam as duas variantes com a mesma notação usada aqui.

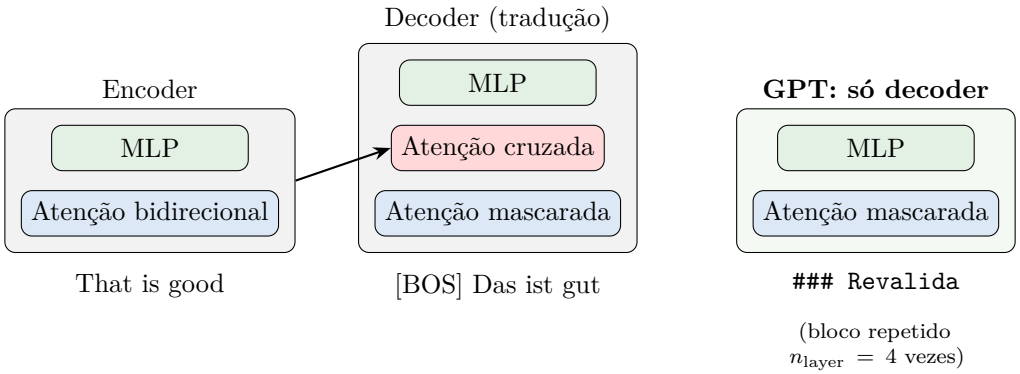


Figura 2: À esquerda, o Transformer de tradução [5]. À direita, o GPT: sem encoder e, portanto, sem atenção cruzada.

5.2 Visão geral e as dimensões

Usamos a notação B para o tamanho do lote (32), T para o comprimento do contexto (até 64), C para a dimensão dos vetores internos, `n_embd` (128), e V para o vocabulário (125). O caminho de um lote pelo modelo é:

$$\begin{aligned}\text{ids} \in \{0..V-1\}^{B\times T} &\xrightarrow{\text{embeddings}} \boldsymbol{X}^{(0)} \in \mathbb{R}^{B\times T\times C} \xrightarrow{\text{4 blocos}} \boldsymbol{X}^{(4)} \in \mathbb{R}^{B\times T\times C} \\ &\xrightarrow{\text{LN, linear}} \text{logits} \in \mathbb{R}^{B\times T\times V}.\end{aligned}$$

A dimensão C fica constante do começo ao fim dos blocos. Isso é o que permite somar a entrada de cada bloco com a saída dele (conexões residuais).

5.3 Embeddings de token e de posição

A *embedding de token* é uma tabela $\boldsymbol{W}_{\text{tok}} \in \mathbb{R}^{V\times C}$: o caractere de id i é representado pela linha i. É equivalente a multiplicar um vetor *one-hot* pela matriz, mas sem fazer a multiplicação. Os valores da tabela são aprendidos, e caracteres que se comportam de forma parecida, como vogais, tendem a ficar com vetores parecidos.

A atenção, sozinha, não sabe a ordem das posições: se embaralharmos as entradas, as saídas se embaralham do mesmo jeito. Por isso somamos uma *embedding de posição* $\boldsymbol{W}_{\text{pos}} \in \mathbb{R}^{T_{\max}\times C}$, também aprendida, como no GPT-2:

$$\boldsymbol{X}_t^{(0)} = \boldsymbol{W}_{\text{tok}}[x_t] + \boldsymbol{W}_{\text{pos}}[t]. \tag{6}$$

O Transformer original [5] usava senos e cossenos fixos em vez de uma tabela aprendida. As duas opções funcionam.

5.4 Atenção causal com várias cabeças

A atenção é a única parte do modelo em que **as posições trocam informação**. Todo o resto (embeddings, MLP, LayerNorm) age em cada posição separadamente.

Consulta, chave e valor. A partir de cada vetor $\boldsymbol{x}_t$, três projeções lineares aprendidas geram:

$$\begin{aligned} \boldsymbol{q}_t &= \boldsymbol{W}_Q \boldsymbol{x}_t \qquad \text{consulta: “o que procuro”,} \\ \boldsymbol{k}_t &= \boldsymbol{W}_K \boldsymbol{x}_t \qquad \text{chave: “o que ofereço”,} \\ \boldsymbol{v}_t &= \boldsymbol{W}_V \boldsymbol{x}_t \qquad \text{valor: “o que passo adiante”.} \end{aligned}$$

A posição t compara sua consulta com a chave de cada posição s pelo produto escalar $\boldsymbol{q}_t \cdot \boldsymbol{k}_s$. Quanto maior o produto, mais a posição s interessa a t. Os produtos viram pesos pelo softmax, e a saída é a média dos valores ponderada por esses pesos. Em forma matricial, para uma cabeça com dimensão d [5]:

$$\text{Atenção}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}} + \boldsymbol{M} \right) \boldsymbol{V}. \tag{7}$$

Por que dividir por $\sqrt{d}$. Suponha que as componentes de $\boldsymbol{q}$ e $\boldsymbol{k}$ sejam independentes, com média 0 e variância 1. Então $\boldsymbol{q} \cdot \boldsymbol{k} = \sum_{i=1}^{d} q_i k_i$ é uma soma de d termos de variância 1, e tem variância d. Com $d = 32$, os produtos teriam desvio padrão $\approx 5{,}7$. Com valores tão espalhados, o softmax fica quase “one-hot”: uma posição recebe quase todo o peso e o gradiente das outras some. Dividir por $\sqrt{d}$ traz a variância de volta para 1.

A máscara causal. A matriz $\boldsymbol{M}$ em (7) tem 0 onde a posição pode olhar ($s \le t$) e $-\infty$ onde não pode ($s > t$). Como $e^{-\infty} = 0$, o softmax dá peso zero ao futuro. Para $T = 4$:

$$\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d}} + \boldsymbol{M} = \begin{pmatrix} a_{11} & -\infty & -\infty & -\infty \\ a_{21} & a_{22} & -\infty & -\infty \\ a_{31} & a_{32} & a_{33} & -\infty \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \xrightarrow{\text{softmax por linha}} \begin{pmatrix} 1 & 0 & 0 & 0 \\ * & * & 0 & 0 \\ * & * & * & 0 \\ * & * & * & * \end{pmatrix}.$$

Cada linha soma 1. A primeira posição só pode olhar para si mesma. Sem a máscara, a posição t poderia ler x_{t+1}, que é exatamente o alvo y_t: prever seria copiar, a perda de treino iria a zero e o modelo não aprenderia nada útil para gerar texto.

Várias cabeças. Em vez de uma atenção com $d = C = 128$, usamos $h = 4$ cabeças com $d = C/h = 32$ cada, em paralelo, com projeções independentes. Cada cabeça pode se especializar: uma olha a letra anterior, outra o início da palavra, outra a abertura de um parêntese. As saídas das cabeças são concatenadas e passam por mais uma projeção linear $\boldsymbol{W}_O$. O custo é o mesmo de uma cabeça grande.

No código. `CausalSelfAttention.forward`

```
q, k, v = self.qkv(x).split(C, dim=2)            # uma camada gera q, k e v
q = q.view(B, T, self.n_head, d).transpose(1, 2)  # (B, n_head, T, d)
# ... o mesmo para k e v
pesos = (q @ k.transpose(-2, -1)) / math.sqrt(d)   # (B, n_head, T, T)
pesos = pesos.masked_fill(self.mascara[:, :, :T, :T] == 0, float("-inf"))
pesos = F.softmax(pesos, dim=-1)                    # futuro recebe peso 0
y = pesos @ v                                        # média ponderada dos
    valores
```

```
y = y.transpose(1, 2).contiguous().view(B, T, C)   # junta as cabeças
return self.resid_dropout(self.proj(y))            # projeção W_O
```

A máscara é uma matriz triangular inferior de uns (`torch.tril`), guardada com `register_buffer`: vai junto para a GPU, mas não é um parâmetro treinável. Escrevemos a fórmula à mão, em vez de usar `F.scaled_dot_product_attention`, para que cada passo de (7) apareça no código.

5.5 MLP, conexões residuais e LayerNorm

MLP. Depois de trocar informação pela atenção, cada posição processa sozinha o que recebeu, com uma rede de duas camadas:

$$\mathrm{MLP}(\boldsymbol{x}) = \boldsymbol{W}_2\ \mathrm{GELU}(\boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2, \qquad \boldsymbol{W}_1 \in \mathbb{R}^{4C\times C},\ \boldsymbol{W}_2 \in \mathbb{R}^{C\times 4C}. \tag{8}$$

A camada interna é 4 vezes mais larga, como em [5]. A ativação GELU [13] é $\mathrm{GELU}(x) = x\,\Phi(x)$, em que Φ é a distribuição acumulada da normal padrão. É uma versão suave da ReLU e é a usada no GPT-2. Sem uma função não linear, as duas camadas lineares equivaleriam a uma só [1, cap. 6]. Um bom resumo da divisão de papéis: **a atenção comunica, o MLP computa**.

Conexões residuais. Cada subcamada *soma* sua saída à entrada, em vez de substituí-la:

$$\boldsymbol{X}' = \boldsymbol{X} + \mathrm{Atenção}(\mathrm{LN}_1(\boldsymbol{X})), \tag{9}$$

$$\boldsymbol{X}'' = \boldsymbol{X}' + \mathrm{MLP}(\mathrm{LN}_2(\boldsymbol{X}')). \tag{10}$$

A ideia vem das ResNets [11]. Na derivada de (9), o termo $\boldsymbol{X}$ contribui com a identidade: $\partial \boldsymbol{X}'/\partial \boldsymbol{X} = \boldsymbol{I} + \dots$. Assim o gradiente sempre tem um caminho direto da perda até as primeiras camadas, mesmo que as subcamadas ainda estejam mal ajustadas. Sem isso, redes profundas treinam muito mal.

LayerNorm. Para cada posição, a LayerNorm [12] padroniza o vetor para média 0 e variância 1 e depois aplica uma escala $\boldsymbol{\gamma}$ e um deslocamento $\boldsymbol{\beta}$ aprendidos:

$$\mathrm{LN}(\boldsymbol{x}) = \boldsymbol{\gamma} \odot \frac{\boldsymbol{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta}, \qquad \mu = \frac{1}{C}\sum_i x_i, \quad \sigma^2 = \frac{1}{C}\sum_i (x_i - \mu)^2. \tag{11}$$

Isso mantém os valores numa escala estável, camada após camada. Usamos a LayerNorm *antes* de cada subcamada ("pre-norm", como no GPT-2 [6]). O artigo original [5] aplicava depois ("post-norm"). O pre-norm deixa o caminho residual limpo e costuma treinar de forma mais estável.

5.6 A saída do modelo

Depois dos 4 blocos, uma última LayerNorm e uma camada linear $\boldsymbol{W}_{\text{out}} \in \mathbb{R}^{V\times C}$ produzem os logits de cada posição. A perda compara os logits com os alvos pela entropia cruzada (2). O PyTorch aplica o softmax dentro de `F.cross_entropy`, de forma numericamente estável.

No código. `GPT.forward`

```
posicoes = torch.arange(T, device=ids.device)
x = self.drop(self.tok_emb(ids) + self.pos_emb(posicoes))  # (B, T, C)
for bloco in self.blocks:
    x = bloco(x)
```

```
logits = self.head(self.ln_f(x))                          # (B, T, V)
perda = F.cross_entropy(logits.view(B * T, -1), alvos.view(B * T))
```

5.7 Contando os parâmetros

Contar à mão é um ótimo exercício para entender a arquitetura. Uma camada linear de n entradas para m saídas tem mn pesos e m vieses; uma LayerNorm de dimensão C tem $2C$ parâmetros ($\boldsymbol{\gamma}$ e $\boldsymbol{\beta}$). Com $C = 128$, $V = 125$, $T_{\max} = 64$:

Parte	Fórmula	Parâmetros
Embedding de token	VC	16.000
Embedding de posição	$T_{\max}C$	8.192
Um bloco:		
LayerNorm 1 e 2	$2 \times 2C$	512
$\boldsymbol{W}_Q, \boldsymbol{W}_K, \boldsymbol{W}_V$ (juntas)	$C \cdot 3C + 3C$	49.536
Projeção $\boldsymbol{W}_O$	$C^2 + C$	16.512
MLP	$(C \cdot 4C + 4C) + (4C \cdot C + C)$	131.712
Total do bloco	$12C^2 + 13C$	198.272
Quatro blocos	4×198.272	793.088
LayerNorm final	$2C$	256
Camada de saída	$CV + V$	16.125
Total		**833.661**

Tabela 3: Contagem de parâmetros do modelo base. O total bate com `modelo.num_parametros()`.

Observe que 95% dos parâmetros estão nos blocos, e cada bloco tem $\approx 12C^2$. Por isso o número de parâmetros cresce com o *quadrado* de C: dobrar `n_embd` multiplica o tamanho dos blocos por 4.

5.8 Inicialização e a perda inicial

Todos os pesos das camadas lineares e das embeddings começam sorteados de uma normal com média 0 e desvio padrão 0,02, e todos os vieses começam em zero, como no GPT-2. Com pesos pequenos, os logits iniciais ficam perto de zero, o softmax fica quase uniforme e a perda fica perto de $\ln V$.

Ela não fica *exatamente* em $\ln V$ porque os logits não são exatamente zero. Se os logits forem aproximadamente independentes com variância σ^2 pequena, uma expansão de segunda ordem dá

$$\mathbb{E}[\mathcal{L}_{\text{inicial}}] \approx \ln V + \tfrac{\sigma^2}{2}.$$

Medindo no modelo recém-criado, os logits têm desvio padrão $\sigma \approx 0{,}23$. A fórmula prevê $4{,}83 + 0{,}23^2/2 \approx 4{,}86$; medimos entre 4,87 e 4,90, conforme o lote. Para comparar, repetimos a medição inicializando os pesos com desvio 1 em vez de 0,02: os logits passam a ter desvio ≈ 11 e a perda inicial vai a ≈ 30. O modelo começaria "confiante e errado" e gastaria os primeiros passos só desfazendo isso.

5.9 Testes de sanidade da arquitetura

O arquivo `tests/test_etapa2.py` verifica quatro propriedades que qualquer implementação correta precisa ter:

1. **Formas:** entrada (B, T) gera logits (B, T, V); sequências maiores que `block_size` são recusadas.
2. **Causalidade:** trocar o token da posição 10 não pode mudar os logits das posições 0 a 9, e precisa mudar os das posições 10 em diante. Esse teste pega qualquer erro na máscara.
3. **Perda inicial:** fica a menos de 0,1 de $\ln V$.
4. **Decorar um lote:** com 200 passos no mesmo lote pequeno, a perda precisa cair abaixo de 0,1. Se não cai, o gradiente não está chegando a alguma parte do modelo. Karpathy [7] recomenda esse teste antes de qualquer treino longo.

Para verificar o entendimento

1. No teste de causalidade, por que os logits das posições 10 em diante *precisam* mudar?
2. Refaça a Tabela 3 para o modelo grande da Seção 8 ($C = 256$, 6 blocos). Confira: 4.819.581.
3. O que aconteceria com o treino se esquecêssemos a embedding de posição?
4. Por que o número de cabeças não muda o número de parâmetros?

6 Etapa 3: o treino

6.1 O laço de treino

Treinar é ajustar os parâmetros $\boldsymbol{\theta}$ (os 833.661 números) para minimizar a perda média no conjunto de treino. Cada passo do `train.py` faz quatro coisas:

```
x, y = lote(dados["train"], cfg, rng, dispositivo)  # 1. sorteia um lote
_, perda = modelo(x, y)                              # 2. forward: logits e perda
otimizador.zero_grad(set_to_none=True)
perda.backward()                                     # 3. backward: gradientes
otimizador.step()                                    # 4. AdamW ajusta os pesos
```

Teoria. Retropropagação e descida do gradiente

O passo 3 calcula $\nabla_{\boldsymbol{\theta}} \mathcal{L}$, a derivada da perda em relação a cada um dos parâmetros. O PyTorch faz isso por *diferenciação automática em modo reverso* (retropropagação): durante o forward, registra o grafo de operações; no backward, aplica a regra da cadeia de trás para a frente, reaproveitando os resultados intermediários [1, seção 6.5]. O custo do backward é da mesma ordem do forward. A forma mais simples de usar o gradiente é a descida estocástica do gradiente (SGD): $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\text{lote}}$. O gradiente de um lote é uma estimativa ruidosa, mas sem viés, do gradiente no conjunto inteiro [1, seção 8.3.1].

O `zero_grad` é necessário porque o PyTorch *acumula* gradientes a cada `backward`: sem zerar, o gradiente de um passo se somaria ao do anterior.

6.2 O otimizador AdamW

Usamos AdamW com taxa de aprendizado fixa $\eta = 10^{-3}$. O Adam [15] mantém, para cada parâmetro, médias móveis exponenciais do gradiente ($\boldsymbol{m}$, o "momento") e do gradiente ao quadrado ($\boldsymbol{v}$):

$$\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1-\beta_1)\,\boldsymbol{g}, \qquad \boldsymbol{v} \leftarrow \beta_2 \boldsymbol{v} + (1-\beta_2)\,\boldsymbol{g}^2,$$
$$\hat{\boldsymbol{m}} = \boldsymbol{m}/(1-\beta_1^k), \qquad \hat{\boldsymbol{v}} = \boldsymbol{v}/(1-\beta_2^k),$$
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \left(\frac{\hat{\boldsymbol{m}}}{\sqrt{\hat{\boldsymbol{v}}} + \epsilon} + \lambda\, \boldsymbol{\theta} \right),$$

com $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$ e k o número do passo. As divisões por $1-\beta^k$ corrigem o viés das médias, que começam em zero. Duas ideias:

- **Momento** ($\hat{\boldsymbol{m}}$): suaviza o ruído dos lotes, como uma bola que ganha velocidade na direção consistente da descida.
- **Passo adaptativo por parâmetro** ($\sqrt{\hat{\boldsymbol{v}}}$): parâmetros com gradientes grandes recebem passos menores, e vice-versa. Isso importa num Transformer, em que embeddings, atenção e LayerNorm têm gradientes de escalas muito diferentes.

O "W" do AdamW [16] é o *decaimento de peso desacoplado*, o termo $\lambda\boldsymbol{\theta}$: a cada passo, os pesos encolhem um pouco na direção de zero, o que é uma forma de regularização [1, seção 7.1]. Usamos o padrão do PyTorch, $\lambda = 0{,}01$.

6.3 Dropout

Com probabilidade $p = 0{,}1$, o dropout [14] zera cada ativação durante o treino e multiplica as que sobram por $1/(1-p)$, para manter a média. Assim, a rede não pode depender de nenhuma unidade específica e aprende representações redundantes. Goodfellow et al. [1, seção 7.12] interpretam o dropout como treinar, ao mesmo tempo, um conjunto enorme de sub-redes que compartilham pesos. No nosso modelo, ele aparece depois das embeddings, nos pesos da atenção, depois da projeção da atenção e no fim do MLP.

6.4 Avaliação durante o treino

A cada 250 passos, a função `estimar_perda` mede a perda média de 100 lotes de treino e 100 lotes de validação. Dois detalhes:

- `modelo.eval()` **desliga o dropout**. Queremos medir o modelo como ele será usado na geração. Depois da medição, `modelo.train()` liga o dropout de novo.
- `@torch.no_grad()` faz o PyTorch não registrar o grafo, o que economiza memória e tempo, porque não haverá backward.

Sempre que a perda de validação melhora, o modelo é salvo em `checkpoints/base.pt`, junto com a configuração e o vocabulário. Assim, o `generate.py` consegue reconstruir tudo só com esse arquivo.

6.5 Resultados do modelo base

Configuração: $C = 128$, 4 cabeças, 4 blocos, contexto 64, lote 32, dropout 0,1, $\eta = 10^{-3}$, 5.000 passos. Na CPU, o treino levou **12,7 minutos**.

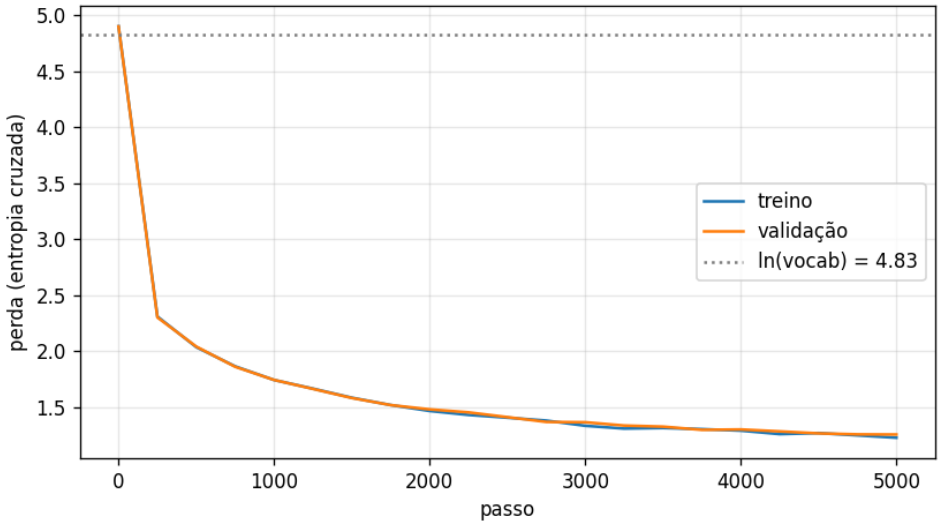


Figura 3: Curva de perda do modelo base. A linha pontilhada é $\ln 125$.

Passo	Treino	Validação	Comparação com os n-gramas da Tabela 2
0	4,90	4,90	pior que o unigrama (3,24): ainda não sabe nada
250	2,31	2,30	melhor que o bigrama (2,39)
1.000	1,74	1,74	entre o trigrama (1,92) e o 4-grama (1,49)
2.500	1,40	1,41	entre o 4-grama (1,49) e o 5-grama (1,22)
5.000	1,23	**1,25**	perto do 5-grama (1,22); acima do 6-grama (1,14)

Tabela 4: Perdas (nats por caractere) ao longo do treino.

Leitura dos resultados.

1. **Sem overfitting.** As curvas de treino e validação ficam juntas até o fim; a diferença final é de 0,03. O modelo não está decorando as questões de treino.

2. **O modelo ainda não convergiu.** A validação ainda caía no passo 5.000. Em 5.000 passos de $32 \times 64 = 2.048$ posições, o modelo viu cerca de 10,2 milhões de posições, ou $\approx 2{,}2$ vezes o conjunto de treino (*épocas*). É pouco treino para um conjunto desse tamanho.

3. **Em bits:** $1{,}25 / \ln 2 \approx 1{,}81$ bits por caractere. Para comparar: Shannon [9] estimou que humanos, prevendo o próximo caractere de textos em inglês, ficam em torno de 1 bit por caractere.

Atenção. O Transformer ainda perde para o 6-grama

Pela Tabela 2, um 6-grama por contagem chega a 1,14 nats, e o nosso Transformer base a 1,25. Isso não quer dizer que a arquitetura seja pior. O Transformer enxerga 64 caracteres para trás, e o 6-grama só 5, mas precisaria de mais treino ou mais capacidade para tirar proveito disso. O 6-grama, por outro lado, "decora" todas as sequências de 6 caracteres do treino de uma vez só, por contagem. Em relatórios, esse tipo de comparação com uma régua simples é essencial: sem ela, 1,25 pareceria um ótimo número. A Seção 8 testa se um modelo maior supera a régua.

Para verificar o entendimento

1. Por que medimos a perda com `modelo.eval()`? O que aconteceria com a perda de treino medida se o dropout continuasse ligado?
2. Quantas épocas o modelo veria com 15.000 passos e a mesma configuração?
3. Se a perda de validação começasse a subir enquanto a de treino continua caindo, o que isso indicaria? E qual checkpoint o `train.py` guardaria?

7 Etapa 4: geração e avaliação

7.1 Amostragem auto-regressiva

O modelo só prevê o próximo caractere. Para escrever um texto, repetimos: passamos o texto até agora (no máximo os últimos 64 caracteres), pegamos os logits da última posição, sorteamos um caractere a partir do softmax, colamos no fim e voltamos ao começo. É a regra da cadeia (1) usada "ao contrário": em vez de calcular a probabilidade de um texto, sorteamos um texto da distribuição.

```
for _ in range(n_novos):
    contexto = ids[:, -block_size:]           # só os últimos 64 caracteres
    logits, _ = modelo(contexto)
    logits = logits[:, -1, :] / temperatura    # só a última posição interessa
    if top_k is not None:                      # corta a cauda improvável
        limite = torch.topk(logits, top_k).values[:, [-1]]
        logits = logits.masked_fill(logits < limite, float("-inf"))
    proximo = torch.multinomial(F.softmax(logits, dim=-1), num_samples=1)
    ids = torch.cat([ids, proximo], dim=1)
```

7.2 Temperatura e top-k

Temperatura. Dividimos os logits por $\tau > 0$ antes do softmax:

$$p_i(\tau) = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}. \tag{12}$$

Com $\tau = 1$, é a distribuição que o modelo aprendeu. Quando $\tau \to 0$, a probabilidade se concentra no maior logit, e o sorteio vira escolha gulosa (sempre o mais provável). Quando $\tau \to \infty$, a distribuição tende à uniforme. Temperatura baixa gera texto seguro e repetitivo; alta gera texto variado e com mais erros.

Top-k. Antes do sorteio, mantemos só os k caracteres mais prováveis e zeramos a probabilidade dos demais [17]. Mesmo quando cada caractere improvável tem probabilidade pequena, juntos eles formam uma "cauda" que é sorteada de vez em quando, e é daí que vêm os erros mais estranhos. Holtzman et al. [18] estudam esse problema e mostram que a escolha gulosa, por outro lado, leva a repetições.

7.3 Resultados

Para avaliar o texto gerado, medimos a fração das palavras geradas (com 3 letras ou mais) que existem no corpus. Cada valor é a média de 5 amostras de 400 caracteres, a partir de `### Revalida`, com a mesma semente para todas as temperaturas:

Temperatura	Palavras reais	Palavras reais com top-$k = 10$
0,5	89%	90%
0,8	80%	83%
1,0	71%	72%
1,2	57%	62%

Tabela 5: Qualidade das amostras do modelo base, por temperatura.

Exemplo com temperatura 0,8:

```
### Revalida 2024 | Terapia coletiva
O paciente de um episódio de relação peso agudo, nas condutas ou a elesão das
enconhecidas. Ao exame físico, mostra-se de albumina tipaneopatose e os pesos em
insuficiências no trace de respectivamente sintomal de diarreia e a fase [...]
(C) resposta utilizada como para episódios dias.
(B) Assinale a apresentar o diagnóstico de processo do venos único o
```

O que as amostras mostram.

- **O formato foi aprendido**: cabeçalho com prova, ano e área; enunciado que começa como caso clínico; alternativas; gabarito.
- **O conteúdo não faz sentido.** As palavras existem, mas as frases não têm significado médico. Com 64 caracteres de contexto (cerca de 10 palavras) e 833 mil parâmetros, o modelo capta ortografia e sintaxe local, não conhecimento.
- **Temperatura baixa repete** expressões ("de saúde de ... de saúde"), como previsto por [18].
- **O top-k ajuda mais nas temperaturas altas**, em que a cauda da distribuição pesa mais.

Atenção. Limites da métrica e da semente

A métrica de palavras reais aceita frases sem sentido formadas por palavras reais; ela mede ortografia, não significado. Além disso, com uma amostra só por temperatura, os números oscilavam demais: a temperatura 0,5 chegou a dar *menos* palavras reais que a 0,8. Tirar a média de 5 amostras resolveu. A perplexidade de validação continua sendo a métrica principal, porque é calculada sobre meio milhão de caracteres.

Para verificar o entendimento

1. Mostre que, com $\tau \to 0$, $p_i(\tau)$ tende a 1 para o maior logit e a 0 para os demais (suponha um único máximo).
2. Qual é a diferença entre top-k com $k = 1$ e temperatura muito baixa?
3. Por que o `generate.py` corta o contexto em `ids[:, -block_size:]`? O que aconteceria sem o corte?

8 Etapa 5: experimentos

Esta etapa tem dois experimentos. O primeiro aumenta o *tamanho* do modelo; o segundo, o *contexto* que ele enxerga. Em cada um, mudamos uma coisa só e mantemos todo o resto igual, para saber a que atribuir a diferença.

8.1 Experimento 1: um modelo maior

Pergunta e hipótese. O modelo base não teve overfitting e ainda perdia para o 6-grama. Quando treino e validação andam juntos, o modelo provavelmente tem *pouca capacidade* para os dados: ele não consegue nem decorar o treino. A hipótese é que um modelo maior chegue a uma perda menor. Kaplan et al. [19] mediram que, em modelos de linguagem, a perda cai de forma previsível (uma lei de potência) conforme crescem o número de parâmetros, a quantidade de dados e o cálculo gasto no treino.

Montagem do experimento. Para a comparação ser justa, mudamos **só o tamanho** do modelo. Passos, lote, contexto, taxa de aprendizado, dropout e semente ficaram iguais aos da base (`configs/grande.json`):

	Base	Grande
`n_embd` (C)	128	256
`n_head` (dimensão por cabeça)	4 (32)	8 (32)
`n_layer`	4	6
Parâmetros	833.661	4.819.581 (5,8×)
Tempo por passo na CPU	≈ 0,15 s	≈ 0,44 s
Tempo total (5.000 passos)	12,7 min	47,6 min

Tabela 6: Configurações comparadas. O restante é igual: contexto 64, lote 32, $\eta = 10^{-3}$, dropout 0,1.

Pela Tabela 3, cada bloco tem $\approx 12C^2$ parâmetros. Dobrar C multiplica cada bloco por 4, e passar de 4 para 6 blocos multiplica por mais 1,5: daí o fator ≈ 6.

Resultados. Além da estimativa com 100 lotes feita durante o treino, medimos os dois modelos finais no conjunto de validação *inteiro* (508.160 posições), para a comparação com os n-gramas ser feita nos mesmos dados:

Modelo	Perda val.	Bits/caractere	Perplexidade	Treino − val.	Palavras reais
Unigrama	3,235	4,67	25,4	–	–
5-grama	1,223	1,76	3,40	–	–
6-grama	1,142	1,65	3,13	–	–
Transformer base	1,249	1,80	3,49	−0,029	80%
Transformer grande	**1,124**	**1,62**	**3,08**	−0,036	**87%**

Tabela 7: Validação inteira. "Treino − val." é a diferença no último passo (negativa: o treino vai melhor). Palavras reais: média de 5 amostras a $\tau = 0{,}8$, com a mesma semente para os dois modelos (`compare.py`).

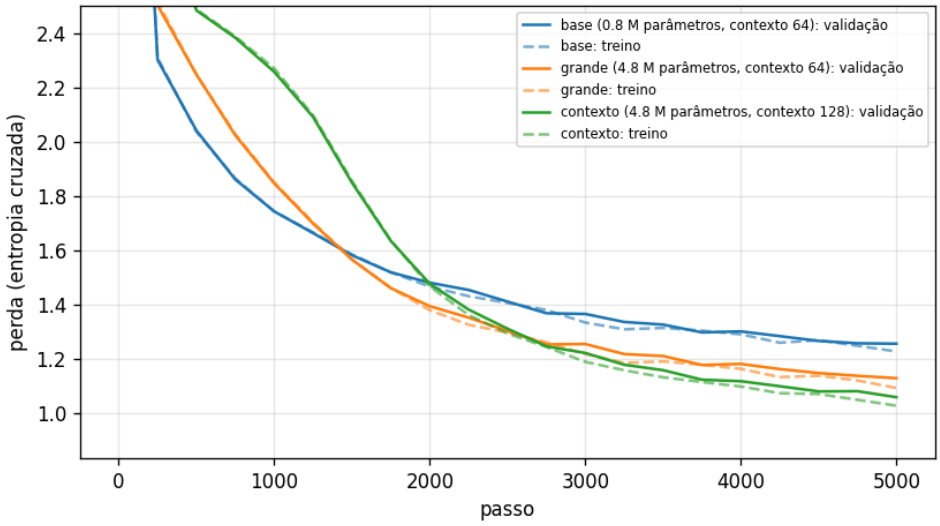


Figura 4: Curvas de perda dos dois modelos. Linha cheia: validação; tracejada: treino. O eixo vertical foi cortado em 2,5 para mostrar a parte final.

Leitura dos resultados.

1. **A hipótese se confirmou.** O modelo grande terminou com perda 0,125 nats menor (1,124 contra 1,249). Em probabilidade média no caractere certo: $e^{-1{,}124} \approx 32{,}5\%$ contra $e^{-1{,}249} \approx 28{,}7\%$.

2. **O Transformer passou do 6-grama, mas por pouco**: 1,124 contra 1,142. Dois detalhes deixam a comparação ainda mais apertada. A suavização do 6-grama foi escolhida olhando a validação, o que o favorece. Por outro lado, na medição do Transformer, as posições do começo de cada janela de 64 caracteres têm pouco contexto, o que o desfavorece.

3. **O modelo grande começa mais devagar.** Até o passo ≈ 1.500, ele fica *acima* da base (Figura 4). Com a mesma taxa de aprendizado, um modelo com mais parâmetros precisa de mais passos até organizar suas representações. Depois disso, ele passa a base e a distância só aumenta.

4. **Com o mesmo tempo de CPU, a base ganha.** Aos 13 minutos, quando a base já tinha terminado com 1,25, o grande estava por volta do passo 1.300, com perda $\approx 1{,}65$. "Qual modelo é melhor" depende da pergunta: com o mesmo número de passos, o grande; com o mesmo orçamento de cálculo curto, o pequeno. Hoffmann et al. [20] estudam exatamente como dividir um orçamento de cálculo entre tamanho do modelo e quantidade de dados.

5. **O começo de uma separação entre treino e validação.** A diferença final passou de $-0{,}029$ para $-0{,}036$, e no passo 5.000 o treino (1,09) está visivelmente abaixo da validação (1,13). Ainda é pequena, mas é o primeiro sinal de que o modelo maior tem capacidade para começar a decorar. Com mais passos, o dropout e o conjunto de validação passam a ser essenciais.

6. **Amostras um pouco melhores.** A taxa de palavras reais subiu de 80% para 87%. O conteúdo continua sem sentido médico:

```
### Revalida 2014
Uma mulher com 44 anos de idade, sem prevenção de mecânio anti-hipertenso. Em
sinais de esclerose impulsiva em espectronente quadro de possa albumina tipa e
pancreática pelo membro inferior de nefrologia à segmentação;
(E) onda endoscopia hormonal do manijor de atenção acessociada a que, está visar
```

Atenção. O que este experimento não mostra

Fizemos **um** treino de cada modelo, com **uma** semente. Parte da diferença pode vir da sorte na inicialização e na ordem dos lotes. Para afirmar com segurança que o grande é melhor, o ideal seria repetir cada treino com 3 sementes e comparar médias e desvios. Aqui, a diferença (0,125 nats) é bem maior que as oscilações entre avaliações vizinhas da mesma curva (cerca de 0,01 a 0,03), o que torna o resultado plausível, mas não o prova.

8.2 Uma correção na forma de medir

Até aqui, a perda na validação inteira foi medida com janelas de 64 caracteres *lado a lado*. Nelas, o primeiro caractere de cada janela é previsto sem contexto nenhum, o segundo com 1 caractere, e assim por diante: em média, cada previsão usa só 32 caracteres de contexto. Isso subestima o modelo e, pior, torna injusta a comparação entre modelos com contextos diferentes.

A medida correta usa uma **janela deslizante**: as janelas andam meio contexto por vez e só a segunda metade de cada uma entra na conta. Assim, todo caractere é previsto com pelo menos metade do contexto do modelo. Esse é o método do `compare.py` a partir daqui. O 6-grama não muda, porque sempre usa os 5 caracteres anteriores.

Modelo	Janelas lado a lado	Janela deslizante
Transformer base	1,249	1,193
Transformer grande	1,124	1,055
6-grama (referência)	1,142	

Tabela 8: A mesma validação, medida das duas formas. Com a medida correta, o modelo grande passa o 6-grama com folga (1,055 contra 1,142), e não "por pouco".

8.3 Experimento 2: mais contexto

Pergunta e hipótese. As amostras do modelo grande pareciam "pedaços colados": cada trecho fazia sentido com a linha anterior, mas o texto perdia o fio. Em 5 amostras de 600 caracteres do modelo grande, medimos que 18% a 46% do texto gerado são trechos de 20 ou mais caracteres idênticos ao corpus, mas nenhum passa de 42 caracteres: são as fórmulas das provas ("Assinale a opção que indica"), não questões copiadas. A explicação provável é o contexto de 64 caracteres, cerca de uma linha. A hipótese: com 128 caracteres, o modelo mantém melhor a estrutura da questão.

Montagem. Igual ao modelo grande, mudando só `block_size` de 64 para 128 (`configs/contexto.json`). A embedding de posição cresce de 64 para 128 linhas, e o modelo passa a ter 4.835.965 parâmetros. Cada passo processa o dobro de caracteres, e o custo da atenção cresce com o quadrado do contexto. O treino levou **120 minutos** na CPU, contra 48 do grande.

Uma medida de coerência. Para testar diretamente se o modelo "lembra" o que escreveu, contamos, dentro de cada questão gerada, os pares de alternativas seguidas e quantos estão em ordem: `(A)` depois `(B)`, `(B)` depois `(C)`. Para acertar, o modelo precisa lembrar a letra da alternativa anterior, que muitas vezes está a mais de 64 caracteres de distância. Geramos 90 textos de 1.000 caracteres por modelo, a partir de `### Revalida`, com temperatura 0,8.

Modelo	Contexto	Perda (deslizante)	Perplexidade	Alternativas em ordem	Tempo
6-grama	5	1,142	3,13	–	–
Base	64	1,193	3,30	–	13 min
Grande	64	1,055	2,87	42,7% (76/178)	48 min
Contexto	**128**	**1,015**	**2,76**	**74,6% (94/126)**	120 min

Tabela 9: Os dois experimentos, com a medida correta de perda.

Resultados.

1. **A perda caiu de 1,055 para 1,015.** É uma melhora menor que a do Experimento 1, mas vem só do contexto: o número de parâmetros é praticamente o mesmo.
2. **A coerência melhorou muito**: de 42,7% para 74,6% das alternativas em ordem. Um teste de diferença entre duas proporções dá $z \approx 5{,}5$; uma diferença desse tamanho por acaso é praticamente impossível. Esse é o efeito que a perda média esconde: a maioria dos caracteres depende só das letras vizinhas, e o contexto longo importa em poucos pontos, justamente os que dão estrutura ao texto.
3. **O modelo com mais contexto começa muito mais devagar.** Até o passo ≈ 1.000, a perda dele fica quase parada entre 2,5 e 2,3, acima dos outros dois. Depois ela cai rápido e passa a do grande por volta do passo 2.750.
4. **O custo é alto**: 2,5 vezes o tempo do grande, para 0,04 nats a menos. Para um orçamento fixo de tempo, pode valer mais treinar o grande por mais passos.

8.4 Próximos experimentos sugeridos

- **Treinar o modelo de contexto 128 por mais passos** (10.000 a 15.000), com taxa de aprendizado decrescente, acompanhando a separação entre treino e validação.
- **Contexto 256**, idealmente na GPU, já que o custo da atenção cresce com o quadrado do contexto.
- **Repetir com sementes diferentes**, para medir a variação entre treinos.
- **Treinar na GPU do Kaggle**, que deve ser dezenas de vezes mais rápida que a CPU para esse tamanho de modelo.

Para verificar o entendimento

1. Usando $\approx 12C^2$ por bloco, estime os parâmetros de um modelo com $C = 384$ e 6 blocos. Quanto tempo por passo você espera na CPU, supondo que o tempo cresce na mesma proporção que os parâmetros?
2. Por que medir no conjunto de validação inteiro é mais confiável que a média de 100 lotes sorteados? Quantas posições cada uma das duas medições usa?
3. Se o treino do modelo grande continuasse até a validação parar de cair, qual checkpoint o `train.py` guardaria? Por que isso protege contra o overfitting?

9 Resumo e próximos passos

1. **Dados.** Limpar o texto (NFC, homóglifos, símbolos equivalentes e raros) reduziu o vocabulário de 144 para 125 caracteres sem mudar o sentido. A separação por questão evita vazamento entre treino e validação.

2. **Arquitetura.** Um Transformer só com decoder: embeddings de token e posição, blocos com atenção causal e MLP, conexões residuais e LayerNorm pre-norm. A perda inicial de 4,90, perto de $\ln 125 = 4{,}83$, e os testes de causalidade e de "decorar um lote" confirmam a implementação.

3. **Treino.** AdamW com dropout, avaliação periódica e checkpoint do melhor modelo. O modelo base chega a 1,25 nats por caractere, sem overfitting e ainda sem ter convergido.

4. **Geração.** O modelo aprende o *formato* das questões, mas não o *conteúdo*. A temperatura controla a troca entre acerto e variedade.

5. **Régua.** Um 6-grama por contagem (1,14) supera o modelo base. Comparar com uma régua simples é o que dá sentido aos números.

6. **Escala.** Com 5,8 vezes mais parâmetros e os mesmos 5.000 passos, a perda caiu para 1,055 (janela deslizante), passando o 6-grama. O modelo maior começa mais devagar, custa 3,7 vezes mais tempo e mostra o primeiro sinal de separação entre treino e validação.

7. **Contexto.** Com 128 caracteres de contexto em vez de 64, a perda caiu para 1,015 e a ordem das alternativas geradas passou de 42,7% para 74,6% de acertos. A perda média esconde parte do ganho: o contexto longo importa em poucos pontos do texto, mas são os que lhe dão estrutura.

Ideias para continuar: treinar por mais passos, com uma taxa de aprendizado que decai ao longo do treino (*cosine schedule*); aumentar o contexto (`block_size` 128 ou 256); repetir os treinos com outras sementes; trocar a tokenização por BPE [10]; treinar na GPU do Kaggle.

Referências

[1] I. Goodfellow, Y. Bengio e A. Courville. *Deep Learning*. MIT Press, 2016. Disponível em `https://www.deeplearningbook.org`.

[2] D. Jurafsky e J. H. Martin. *Speech and Language Processing*, 3.ª ed. (rascunho). Disponível em `https://web.stanford.edu/~jurafsky/slp3/`.

[3] T. M. Cover e J. A. Thomas. *Elements of Information Theory*, 2.ª ed. Wiley, 2006.

[4] C. M. Bishop e H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024.

[5] A. Vaswani et al. Attention Is All You Need. *NeurIPS*, 2017.

[6] A. Radford et al. Language Models are Unsupervised Multitask Learners (GPT-2). OpenAI, 2019.

[7] A. Karpathy. *Let's build GPT: from scratch, in code, spelled out* (vídeo) e repositório nanoGPT. 2023. `https://github.com/karpathy/nanoGPT`.

[8] Y. Bengio, R. Ducharme, P. Vincent e C. Jauvin. A Neural Probabilistic Language Model. *JMLR*, 3:1137–1155, 2003.

[9] C. E. Shannon. Prediction and Entropy of Printed English. *Bell System Technical Journal*, 30(1):50–64, 1951.

[10] R. Sennrich, B. Haddow e A. Birch. Neural Machine Translation of Rare Words with Subword Units. *ACL*, 2016.

[11] K. He, X. Zhang, S. Ren e J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.

[12] J. L. Ba, J. R. Kiros e G. E. Hinton. Layer Normalization. arXiv:1607.06450, 2016.

[13] D. Hendrycks e K. Gimpel. Gaussian Error Linear Units (GELUs). arXiv:1606.08415, 2016.

[14] N. Srivastava et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *JMLR*, 15:1929–1958, 2014.

[15] D. P. Kingma e J. Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015.

[16] I. Loshchilov e F. Hutter. Decoupled Weight Decay Regularization. *ICLR*, 2019.

[17] A. Fan, M. Lewis e Y. Dauphin. Hierarchical Neural Story Generation. *ACL*, 2018.

[18] A. Holtzman et al. The Curious Case of Neural Text Degeneration. *ICLR*, 2020.

[19] J. Kaplan et al. Scaling Laws for Neural Language Models. arXiv:2001.08361, 2020.

[20] J. Hoffmann et al. Training Compute-Optimal Large Language Models. *NeurIPS*, 2022.

[21] HealthQA-BR. Conjunto de dados no Hugging Face: `https://huggingface.co/datasets/Larxel/healthqa-br`. Licença CC-BY 4.0.